

tempssm: State-space modeling for temperature time series in R

Version 0.3.0

Akira S. Hirao, Shinya Baba, and Momoko Ichinokawa

2026-07-28

Summary

tempssm is an R package for state-space modeling of environmental temperature time series, including air and water temperature observations. It provides a practical framework for assessing how long-term trend, seasonal variation, autoregressive dependence, and optional exogenous effects contribute to observed temporal variation. The package facilitates the application of linear Gaussian state-space models estimated by Kalman filtering and smoothing, using the **KFAS** package as the computational backend (Helske, 2017).

Key Features

- Designed for environmental temperature time series with arbitrary seasonal frequencies; currently validated primarily on monthly data
- Estimates latent states using linear Gaussian state-space models combined with Kalman filtering and smoothing
- Models temperature dynamics as a sum of interpretable latent components, including long-term trend, seasonal variation, autoregressive dependence, and optional exogenous effects
- Allows users to specify an arbitrary order of the autoregressive component (default: AR(1))
- Implements time-series cross-validation for model evaluation

Input Data Format

The core modeling functions in **tempssm** expect input time series to be supplied as R **ts** objects. The **ts** class is base R's standard format for regularly spaced time series (see the **stats::ts** documentation: <https://search.r-project.org/R/refmans/stats/html/ts.html>). The package also provides utility functions for converting common tabular and observational data formats into **ts** objects before model fitting.

Prior Art and Scope

tempssm provides a domain-focused workflow for analyzing temperature time series with linear Gaussian state-space models. It brings together model construction, component extraction, uncertainty summaries, residual diagnostics, visualization, and time-series cross-validation in a single R package interface tailored to temperature applications.

The package builds on established statistical methodology, including linear Gaussian state-space modeling, Kalman filtering, and Kalman smoothing. Model estimation is handled through the **KFAS** package, which provides a general framework for state-space models in R.

The initial implementation was adapted from the supplementary code provided by Baba (2024), accompanying Baba et al. (2024), which analyzed sea temperature trends using a linear Gaussian state-space model. The supplementary code is publicly available at:

<https://github.com/logics-of-blue/sea-temperature-trend-jogashima>

Compared with that prior implementation, **tempssm** extends the workflow into a reusable R package interface with input validation, documented S3 methods, tests, diagnostics, cross-validation utilities, and examples for broader temperature time-series analysis.

The next section gives a brief model overview. A separate model specification vignette describes the mathematical formulation in more detail.

Model Overview

tempssm represents temperature variation with a linear Gaussian state-space model composed of interpretable latent components: a long-term trend, seasonal variation, autoregressive dependence, and optional exogenous effects. The model is estimated with KFAS, and **tempssm()** returns both filtering and smoothing estimates. Unless otherwise stated, the summaries, diagnostics, and plots in this vignette use smoothed state estimates.

The full mathematical specification, including the observation equation, state decomposition, seasonal constraint, autoregressive component, exogenous component, parameter count, and estimation procedure, is described in the separate model specification document, available as a PDF in the package repository:

<https://github.com/akihirao/tempssm/blob/main/vignettes/model-specification.pdf>

Setup

Load **tempssm** before running the examples below.

```
## Set libraries
library(tempssm)
```

The objective of this practice is to demonstrate the basic application of a linear Gaussian state-space model to a univariate temperature time series. This example introduces the basic workflow for fitting a **tempssm** model, inspecting the summary output, plotting latent components, checking residual diagnostics, and making short-term predictions.

Example Dataset

This tutorial uses **nottem**, a monthly air temperature time series included with base R.

- **Dataset:** Average monthly air temperature at Nottingham, England
- **Format:** Univariate **ts** object
- **Frequency:** 12 (monthly)
- **Original unit:** Degrees Fahrenheit
- **Period:** January 1920 to December 1939

The original **nottem** series is stored in degrees Fahrenheit. The examples below convert it to degrees Celsius before model fitting, so that the units are consistent with the plotting labels used throughout this package.

```
data("nottem", package = "datasets") # load a base R ts object
nottem_c <- (nottem - 32) * 5 / 9
head(nottem_c)
```

```
##           Jan           Feb           Mar           Apr           May           Jun
## 1920  4.777778  4.888889  6.888889  8.166667 12.277778 14.722222
```

```
summary(nottem_c)
```

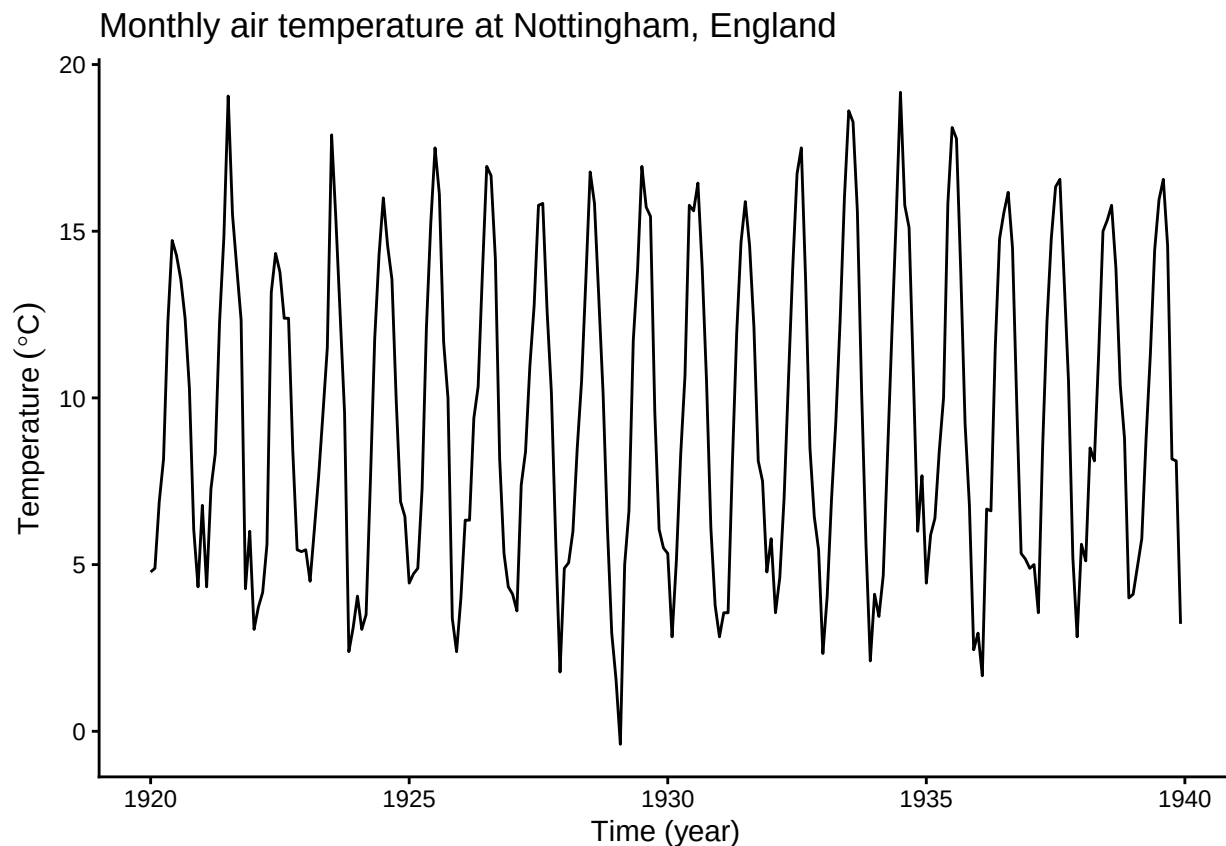
```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.3889  5.3056   8.5278   9.4664 13.8889 19.1667
```

This dataset contains no missing observations. In general, `tempssm()` can retain missing values in the response temperature series and treat them as unobserved responses during Kalman filtering and smoothing.

Plot the Time Series

We begin by visualizing the monthly air temperature time series to examine its overall structure, including apparent trends, seasonal variability, and whether missing observations are present.

```
plt_nottem <- forecast::autoplot(nottem_c) +  
  ggplot2::labs(  
    y = expression(Temperature ~ (degree * C)),  
    x = "Time (year)"  
  ) +  
  ggplot2::ggtitle("Monthly air temperature at Nottingham, England") +  
  ggplot2::theme_classic()  
  
plot(plt_nottem)
```



The overall mean air temperature is approximately 9.5 °C, and a clear annual seasonal pattern is visible. The series contains no missing observations. Although some year-to-year variation is evident, a long-term trend is difficult to assess from the raw time series alone because the seasonal cycle dominates the visual pattern.

Fit a State-Space Model

When a `ts` object containing temperature time-series data (here, `nottem_c`) is passed to the core function `tempssm()`, model construction and parameter estimation are performed together. The returned S3 object of

class `tempssm` (here, `res`) stores the filtering and smoothing estimates, as well as the constructed model and input data. By default, `tempssm()` fits a first-order autoregressive model.

```
# model with first-order autoregressive component
res <- tempssm(nottem_c) # AR(1), the default model
summary(res)
```

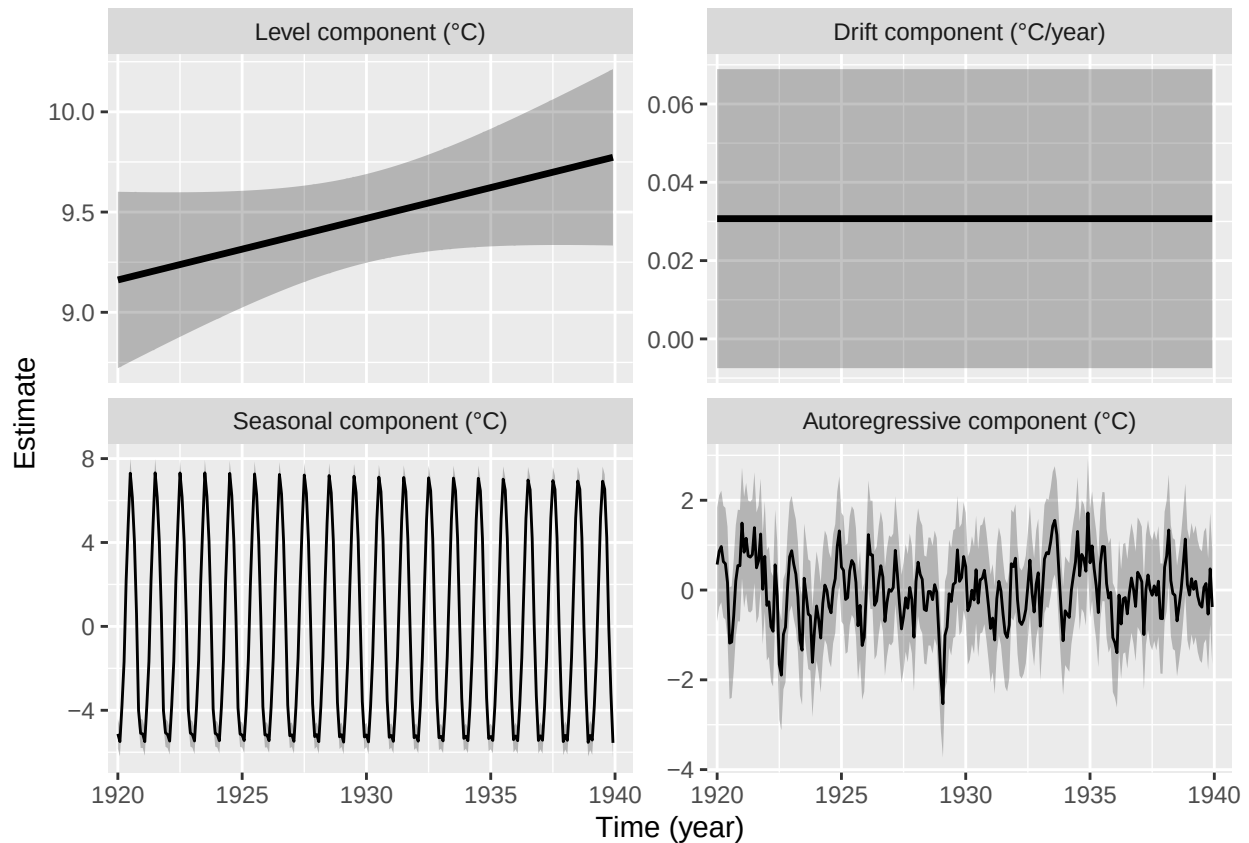
```
## tempssm summary
## -----
## Call:
## tempssm(temp_data = nottem_c)
##
## Model fit:
##   Likelihood type: marginal
##   Log-likelihood : -369.6
##   k               : 5
##   Diffuse states  : 13
##   Converged       : TRUE
##
## Variance parameters:
##   Observation (H): 0.7098195
##   State (Q trend): 9.981401e-16
##   State (Q season): 0.007209047
##   State (Q ar): 0.6814145
##
## Components of auto-regression:
##   Order of AR: 1
##   Coefficient of AR1: 0.4629583
```

First, confirm from the summary output that the model has converged (**Converged: TRUE**). The summary also reports the log-likelihood, parameter count (**k**), likelihood type, and number of diffuse initial states. The estimated parameters include the observation error variance (**H**) and the process error variances for the long-term trend (**Q trend**), seasonal component (**Q season**), and autoregressive component (**Q ar**), as well as the autoregressive coefficient (**AR1** in the default model). See the detailed manual for extracting and using these quantities directly; its location is listed at the end of this quick tutorial.

Plot Model Components

We visualize the estimated long-term evolution of temperature levels and their rates of change (drift) by extracting the corresponding latent components from the state-space model. Additionally, seasonal variability and autoregressive dependence are plotted out, allowing the underlying trend behavior to be examined more clearly.

```
# plot all components at once
plot(res)
```



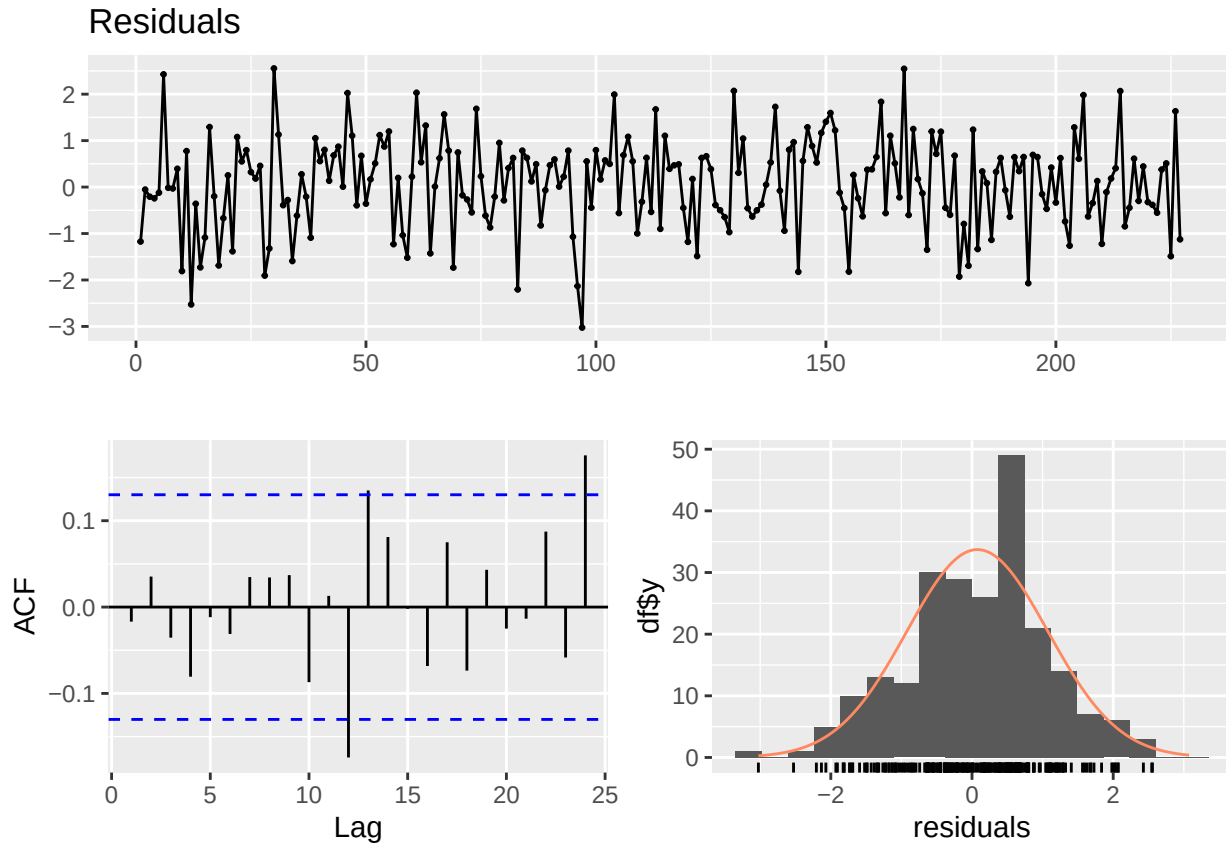
The level component summarizes gradual changes in the underlying temperature level after removing the dominant seasonal pattern. In this example, the estimated long-term level changes only moderately over the study period. The drift component remains close to zero for much of the series, suggesting that the long-term rate of change is small compared with the seasonal variation. The seasonal component captures the recurring annual cycle, while the autoregressive component represents shorter-term departures from the trend and seasonal pattern. The shaded gray areas represent 95% confidence intervals for the estimated latent states, illustrating the uncertainty associated with each estimated component.

The standard plotting interface is `plot(res)`. The explicit helper `plot_tempssm_components(res)` produces the same component plot and can be useful in scripts where a descriptive function name is preferred. The ggplot2-style S3 interface `ggplot2::autoplot(res)` is also available. These interfaces return a faceted `ggplot` object, so selected components can be stored and customized with standard ggplot2 layers, for example, `plot_tempssm_components(res, component = c("level", "drift")) + ggplot2::theme_bw()`.

Check Residual Diagnostics

The package provides diagnostic tools for checking whether the fitted model has left notable structure in the residuals. In particular, residual time-series plots, residual autocorrelation, residual distributions, and Ljung-Box tests can be used to assess remaining temporal dependence and departures from the Gaussian error assumption.

```
r <- get_tempssm_residuals(res)
lb_lag <- frequency(res$temp_data)
forecast::checkresiduals(r, lag = lb_lag, test = "LB")
```



```
##
##  Ljung-Box test
##
## data:  Residuals
## Q* = 12.494, df = 12, p-value = 0.4068
##
## Model df: 0.   Total lags used: 12
```

Here, `get_tempsm_residuals()` explicitly extracts the standardized recursive residuals from the fitted model. `forecast::checkresiduals()` then displays the residual time series, residual autocorrelation plot (ACF plot), and residual frequency distribution, together with a Ljung-Box test. The lag is set to the seasonal frequency of the input data; for monthly data, this uses lag 12. These plots and the test result should be checked for any notable residual patterns. In this example, the Ljung-Box test indicated no significant residual autocorrelation up to lag 12 ($P > 0.05$).

For tabular residual diagnostic summaries, see `diagnose_residuals()` in the detailed manual.

For repeated checks across many fitted models, the convenience wrapper `plot_tempsm_residual_diagnostics(res)` provides the same type of residual diagnostic plot.

Extract Components

The long-term trend component and its rate of change (drift) can be extracted as ts objects as follows.

```
# Smoothing estimates
alpha_hat <- res$kfs$alphahat
head(alpha_hat)
```

```
##           level      slope sea_dummy1 sea_dummy2 sea_dummy3 sea_dummy4
```

```
## Jan 1920 9.161316 0.002559957 -5.126035 -5.098955 -4.008836 0.616990
## Feb 1920 9.163876 0.002559957 -5.493523 -5.126035 -5.098955 -4.008836
## Mar 1920 9.166436 0.002559957 -3.751737 -5.493523 -5.126035 -5.098955
## Apr 1920 9.168996 0.002559957 -1.684483 -3.751737 -5.493523 -5.126035
## May 1920 9.171556 0.002559957 2.164383 -1.684483 -3.751737 -5.493523
## Jun 1920 9.174116 0.002559957 4.880230 2.164383 -1.684483 -3.751737
##          sea_dummy5 sea_dummy6 sea_dummy7 sea_dummy8 sea_dummy9 sea_dummy10
## Jan 1920 4.048339 6.152027 7.301599 4.880230 2.164383 -1.684483
## Feb 1920 0.616990 4.048339 6.152027 7.301599 4.880230 2.164383
## Mar 1920 -4.008836 0.616990 4.048339 6.152027 7.301599 4.880230
## Apr 1920 -5.098955 -4.008836 0.616990 4.048339 6.152027 7.301599
## May 1920 -5.126035 -5.098955 -4.008836 0.616990 4.048339 6.152027
## Jun 1920 -5.493523 -5.126035 -5.098955 -4.008836 0.616990 4.048339
##          sea_dummy11 arima1
## Jan 1920 -3.751737 0.5680923
## Feb 1920 -1.684483 0.8654509
## Mar 1920 2.164383 0.9698171
## Apr 1920 4.880230 0.6325017
## May 1920 7.301599 0.5862658
## Jun 1920 6.152027 0.1679531
```

```
# Smoothing estimate of level component
level_ts <- get_level_ts(res)

# Smoothing estimate of drift component
drift_ts <- get_drift_ts(res)

# Average drift rate per year across the full period
mean_drift_year <- mean(drift_ts)
mean_drift_year
```

```
## [1] 0.03071949
```

Average annual change in air temperature is approximately 0.03 °C.

Make Short-Term Predictions

A fitted `tempssm` object can also be passed to `predict()` to obtain short-term predictions. By default, `predict(res)` returns a one-step-ahead prediction beyond the end of the observed series. This is useful for visual checks of how the fitted model extrapolates the estimated level, seasonal, and autoregressive components.

```
pred_1 <- predict(res)
pred_1
```

```
##          Jan
## 1940 4.407497
```

Predictions for multiple future time points can be requested by setting the `n.ahead` argument.

```
pred_12 <- predict(res, n.ahead = 12)
pred_12
```

```
##          Jan          Feb          Mar          Apr          May          Jun          Jul
## 1940 4.407497 4.276443 5.953507 8.290143 11.607228 14.951480 16.710901
##          Aug          Sep          Oct          Nov          Dec
## 1940 16.336418 14.024846 9.709648 6.633447 4.250628
```

These predictions should be interpreted as model-based extrapolations rather than definitive forecasts. Uncertainty generally increases as the prediction horizon becomes longer, and long-horizon predictions can be sensitive to model assumptions about trend, seasonality, and autoregressive dependence.

Further Reading

The examples above illustrate the basic univariate workflow for fitting, diagnosing, visualizing, and making short-term predictions from a `tempssm` model. The detailed manual extends this workflow to exogenous variables, additional model diagnostics, and time-series cross-validation:

https://github.com/akihirao/tempssm/blob/main/tools/manual/tempssm_manual.pdf